UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM, NAGERCOIL - 629004

RECORD NOTE BOOK

COMPILER DESIGN LABORATORY – CS3501

Register No :

Name :

Year/Semester :

Department :

UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM, NAGERCOIL - 629004

Register No:

Certified that, this is the bonafide record of work done by Mr/Ms. ..... of V Semester in Computer Science and Engineering of this college, in the Compiler Design Laboratory (CS3501) during academic year 2025-2026 in partial fulfillment of the requirements of the B.E Degree course of the Anna University Chennai.

Staff-in-charge **Head of the Department**

This record is submitted for the University Practical Examination held on .....

Internal Examiner **External Examiner**

LIST OF EXPERIMENTS

S.No	Date	Name of the Experiments	Page Number	Sign

S.No	Date	Name of the Experiments	Page Number	Sign

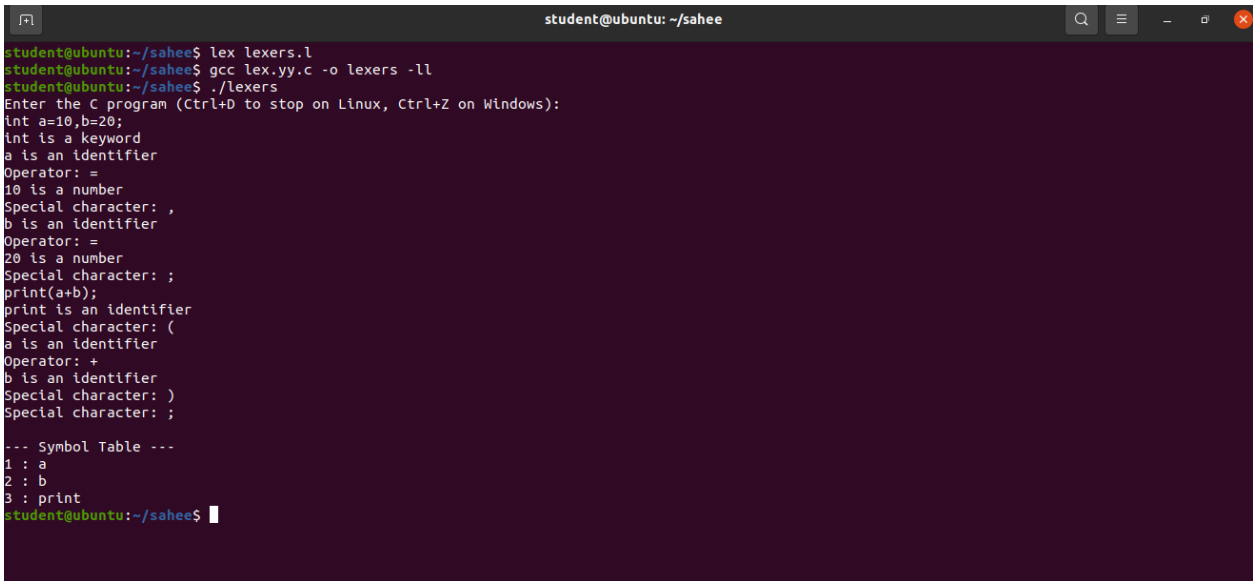
Program:

lexers.l

```
%{
#include <stdio.h>
#include <string.h>
char symtab[100][50];
int symcount = 0;
int isKeyword(char *str) {
    char *keywords[] = {
        "int", "float", "if", "else", "while", "for", "do",
        "break", "continue", "return", "void", "char", "double",
        "long", "short", "switch", "case", "default", "goto", "sizeof"
    };
    int i;
    for (i = 0; i < sizeof(keywords) / sizeof(keywords[0]); i++) {
        if (strcmp(str, keywords[i]) == 0) return 1;
    }
    return 0;
}
void insertSym(char *str) {
    for (int i = 0; i < symcount; i++) {
        if (strcmp(symtab[i], str) == 0) return;
}strcpy(symtab[symcount++], str);
}
%}
%%
"/*"([^*]|\*+[^*/])*\*+"/"      { printf("Comment: %s\n", yytext); }
[0-9]+                          { printf("%s is a number\n", yytext); }
"= ="|"!="|"<="|">="|"<"|">"    { printf("Operator: %s\n", yytext); }
"="|"+"|"-"|"*"|"/"|"%"         { printf("Operator: %s\n", yytext); }
"{"|"}"|"("|")"|"["|"]"|";"|"," {
    printf("Special character: %s\n", yytext);
```

```
}
[a-zA-Z_][a-zA-Z0-9_]*        {
    if (isKeyword(yytext))
        printf("%s is a keyword\n", yytext);
    else {
        printf("%s is an identifier\n", yytext);
        insertSym(yytext);
    }
}
[ \t\n]+                    ;
.                           { printf("Unknown token: %s\n", yytext); }
%%
int main() {
    printf("Enter the C program (Ctrl+D to stop on Linux, Ctrl+Z on Windows):\n");
    yylex();
     printf("\n--- Symbol Table ---\n");
    for (int i = 0; i < symcount; i++) {
        printf("%d : %s\n", i+1, symtab[i]);
    }
    return 0;
}
int yywrap() {
    return 1;
}
```

Output:

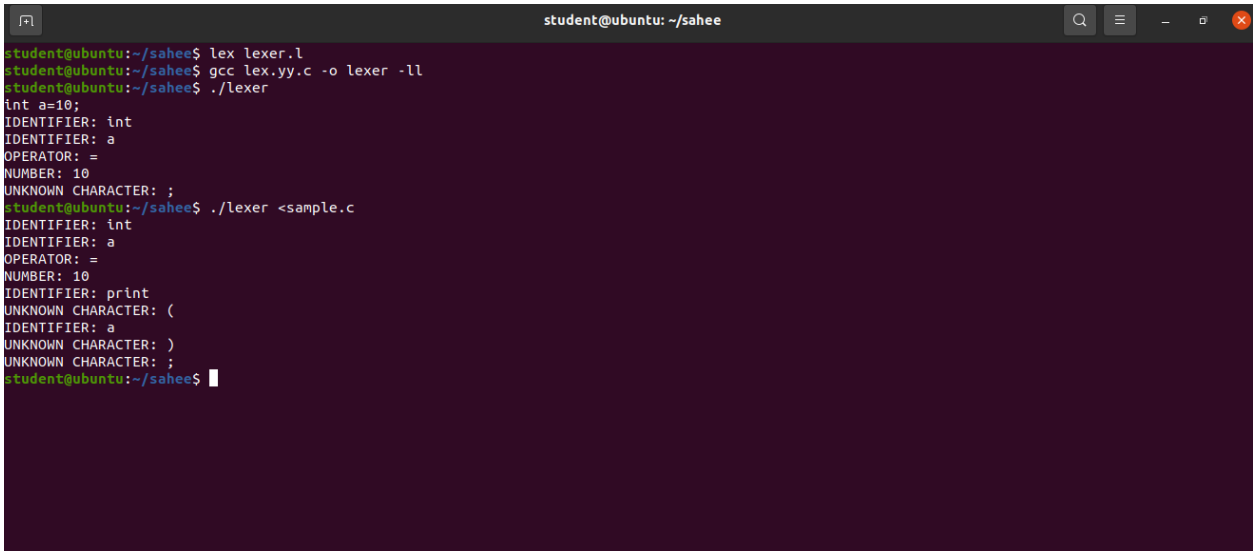

student@ubuntu: ~/sahee
student@ubuntu:~/sahee$ lex lexers.l
student@ubuntu:~/sahee$ gcc lex.yy.c -o lexers -ll
student@ubuntu:~/sahee$ ./lexers
Enter the C program (Ctrl+D to stop on Linux, Ctrl+Z on Windows):
int a=10,b=20;
int is a keyword
a is an identifier
Operator: =
10 is a number
Special character: ,
b is an identifier
Operator: =
20 is a number
Special character: ;
print(a+b);
print is an identifier
Special character: (
a is an identifier
Operator: +
b is an identifier
Special character:)
Special character: ;
--- Symbol Table ---
1 : a
2 : b
3 : print
student@ubuntu:~/sahee$

Program:

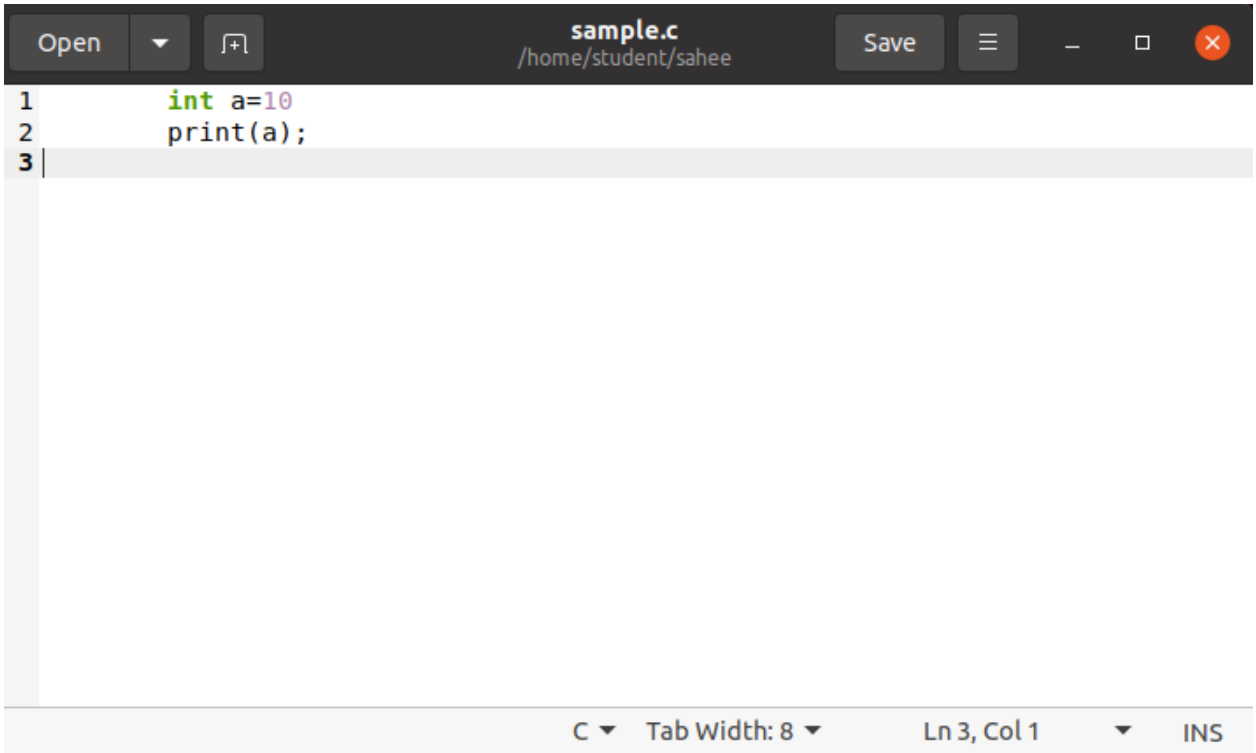
lexer.l

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

IDENTIFIER [a-zA-Z_][a-zA-Z0-9_]*
NUMBER     [0-9]+
WHITESPACE [ \t\n]+

%%
"if"        { printf("KEYWORD: if\n"); }
"else"      { printf("KEYWORD: else\n"); }
"while"     { printf("KEYWORD: while\n"); }
{IDENTIFIER} { printf("IDENTIFIER: %s\n", yytext); }
{NUMBER}     { printf("NUMBER: %s\n", yytext); }
"+"|"-"|"*"|"/"|"=" { printf("OPERATOR: %s\n", yytext); }
{WHITESPACE} { /* Ignore whitespace */ }
.            { printf("UNKNOWN CHARACTER: %s\n", yytext); }
%%

int main() {
    yylex(); // Start lexical analysis
    return 0;
}
```

Output:



Sample.c



Program:

Ex3a.l

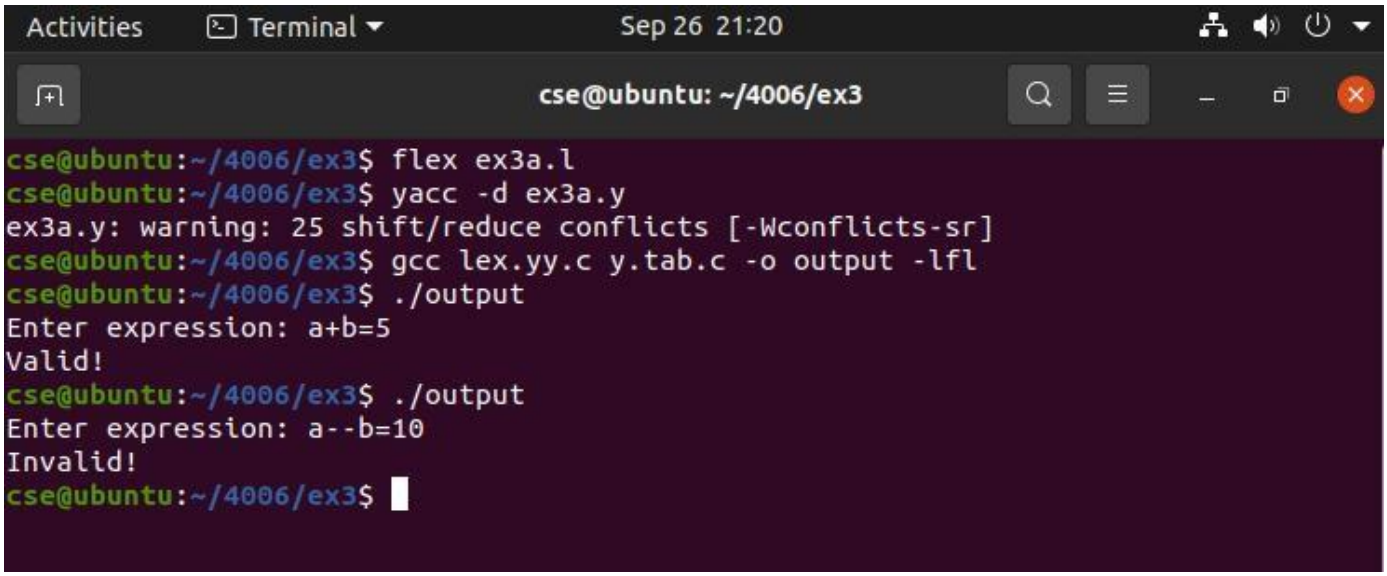
```
%{
#include "y.tab.h"
%}

%%

[0-9]+          { return NUMBER; }
[a-zA-Z][a-zA-Z0-9]*   { return ID; }
[+\-*/]         { return yytext[0]; }
[ \t\n]         ;   /* ignore spaces and newlines */
.               { return yytext[0]; }

%%

int yywrap() { return 1; }
```

Ex3a.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
int yyerror(const char *s);
%}

%token NUMBER ID

%%
```

```
expr:
      expr '+' expr
    | expr '-' expr
    | expr '*' expr
    | expr '/' expr
    | expr '=' expr
    | NUMBER
    | ID
    ;

%%

int main() {
    printf("Enter expression: ");
    if (yyparse() == 0) {   // parsing success
        printf("Valid!\n");
    }
    return 0;
}

int yyerror(const char *s) {
    printf("Invalid!\n");
    exit(1);
}
```

Output:


Activities
Terminal
Sep 26 21:20
cse@ubuntu: ~/4006/ex3
cse@ubuntu:~/4006/ex3$ flex ex3a.l
cse@ubuntu:~/4006/ex3$ yacc -d ex3a.y
ex3a.y: warning: 25 shift/reduce conflicts [-Wconflicts-sr]
cse@ubuntu:~/4006/ex3$ gcc lex.yy.c y.tab.c -o output -lfl
cse@ubuntu:~/4006/ex3$ ./output
Enter expression: a+b=5
Valid!
cse@ubuntu:~/4006/ex3$ ./output
Enter expression: a--b=10
Invalid!
cse@ubuntu:~/4006/ex3$

Program:

Ex3b.l:

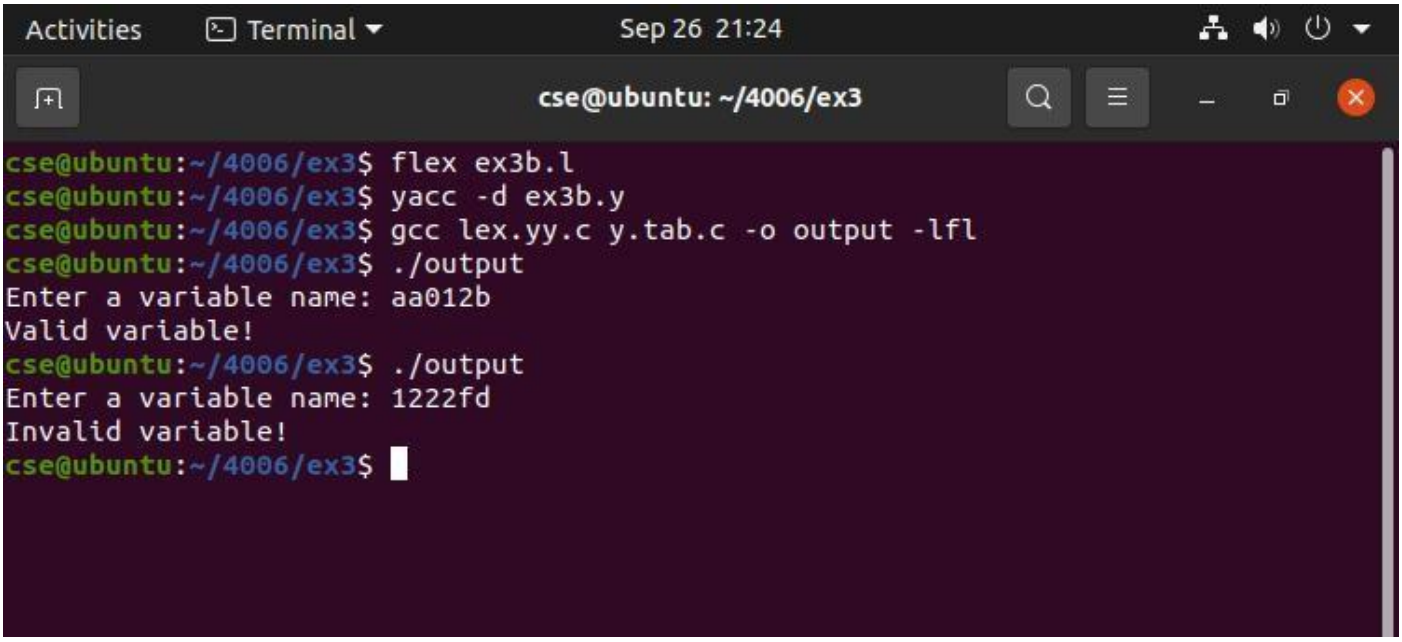
```
%{
#include "y.tab.h"
%}

%%

[a-zA-Z][a-zA-Z0-9]*   { return ID; }
\n                     { return '\n'; }
.                      { return yytext[0]; }

%%

int yywrap() { return 1; }
```

Ex3b.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
int yyerror(const char *s);
%}

%token ID

%%

line:
   ID '\n'    { printf("Valid variable!\n"); }
```

```
  | '\n'       { /* ignore empty line */ }
  ;

%%

int main() {
    printf("Enter a variable name: ");
    yyparse();
    return 0;
}

int yyerror(const char *s) {
    printf("Invalid variable!\n");
    return 0;
}
```

Output:



Program:

Exp3c.l:

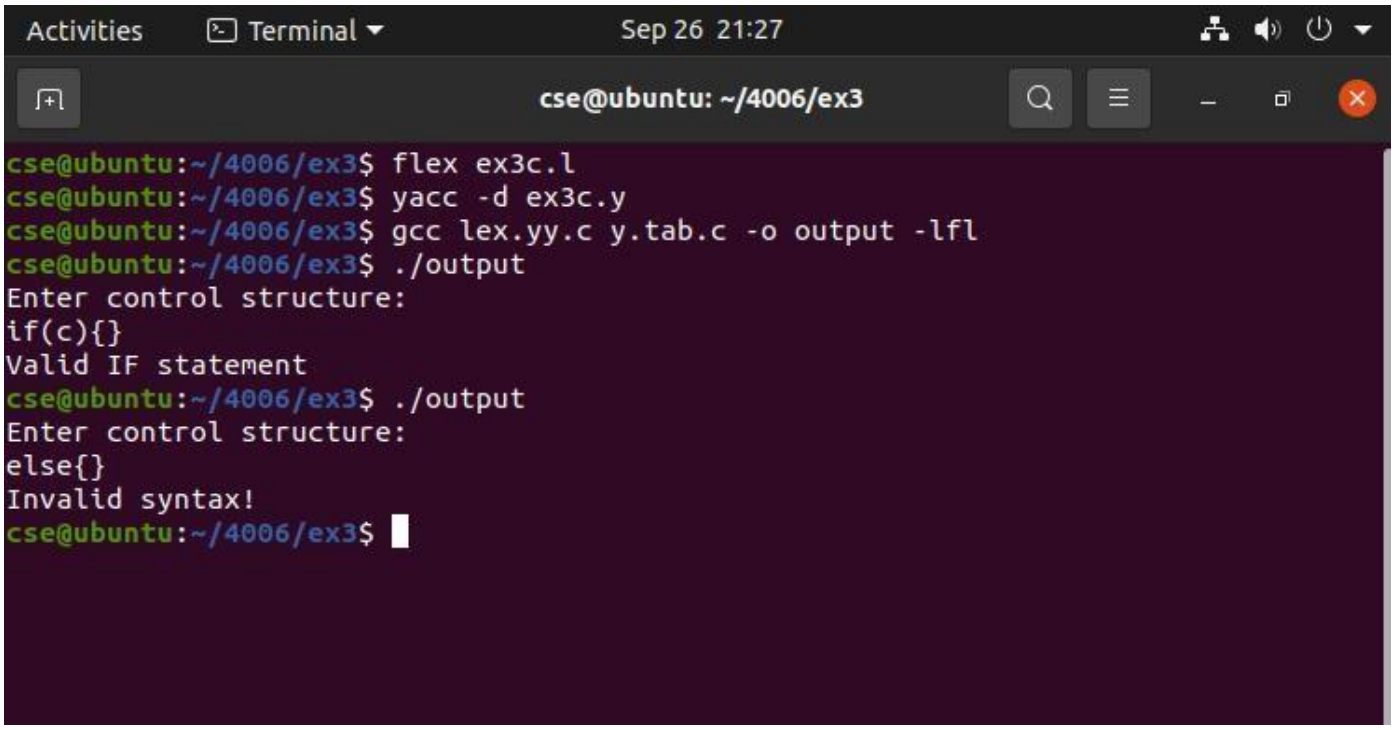
```
%{
#include "y.tab.h"
%}

%%

"for"          { return FOR; }
"while"        { return WHILE; }
"if"           { return IF; }
"else"         { return ELSE; }
"switch"       { return SWITCH; }
"case"         { return CASE; }
"default"      { return DEFAULT; }

"("            { return LPAREN; }
")"            { return RPAREN; }
"{"            { return LBRACE; }
"}"            { return RBRACE; }
";"            { return SEMICOLON; }
":"            { return COLON; }

[a-zA-Z_][a-zA-Z0-9_]*  { return ID; }
[0-9]+                 { return NUMBER; }

[ \t\n]+       ; /* ignore spaces */
.              { return yytext[0]; }

%%
int yywrap() { return 1; }
```

Exp3c.y:

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
int yyerror(const char *s);
%}

%token FOR WHILE IF ELSE SWITCH CASE DEFAULT
%token ID NUMBER LPAREN RPAREN LBRACE RBRACE SEMICOLON COLON

%%

stmt:
      FOR LPAREN ID SEMICOLON ID SEMICOLON ID RPAREN LBRACE RBRACE
        { printf("Valid FOR loop\n"); }
    | WHILE LPAREN opt_id RPAREN LBRACE RBRACE
        { printf("Valid WHILE loop\n"); }
    | IF LPAREN opt_id RPAREN LBRACE RBRACE
        { printf("Valid IF statement\n"); }
    | IF LPAREN opt_id RPAREN LBRACE RBRACE ELSE LBRACE RBRACE
        { printf("Valid IF-ELSE statement\n"); }
    | SWITCH LPAREN opt_id RPAREN LBRACE CASE NUMBER COLON DEFAULT COLON
RBRACE
        { printf("Valid SWITCH statement\n"); }
    ;

opt_id:
    | ID
    ;
```

```
%%

int main() {
    printf("Enter control structure:\n");
    yyparse();
    return 0;
}

int yyerror(const char *s) {
    printf("Invalid syntax!\n");
    return 0;
}
```

Output:



Program:

Exp3d.l:

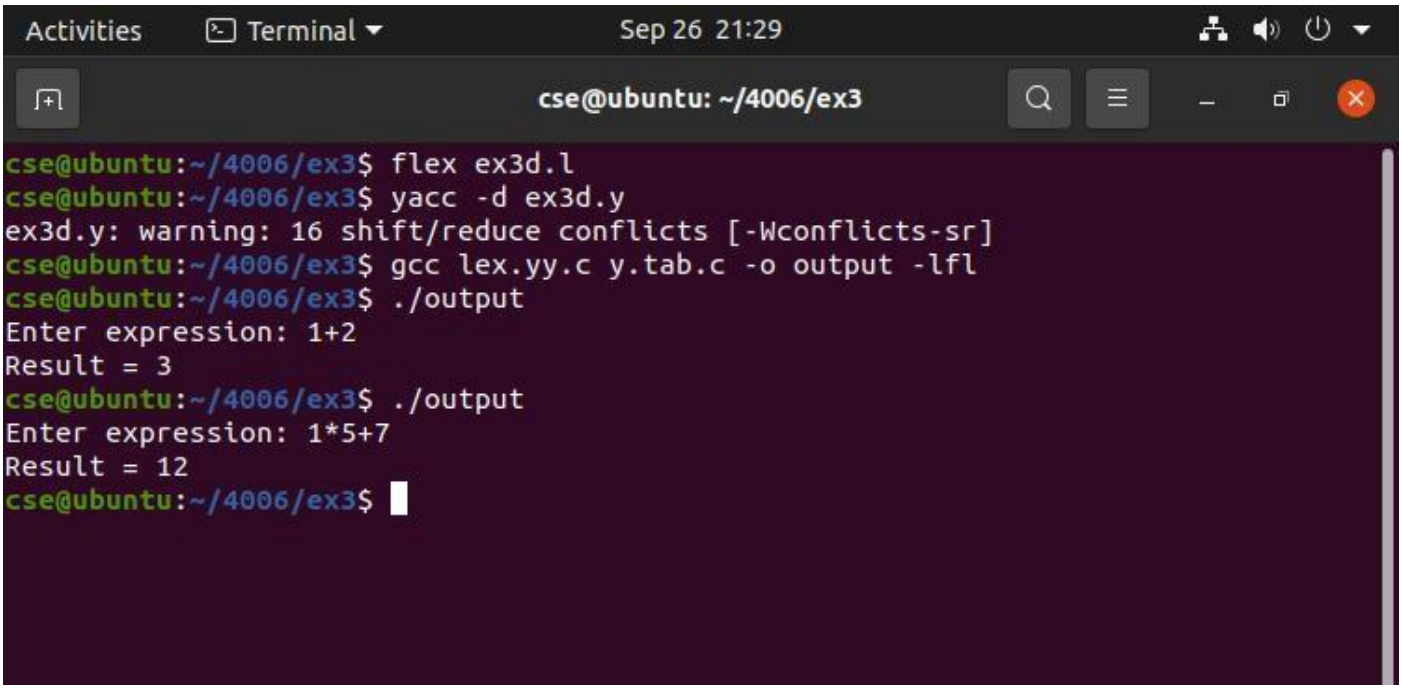
```
%{
#include "y.tab.h"
%}

%%

[0-9]+          { yylval = atoi(yytext); return NUMBER; }
[+\-*/()]       { return yytext[0]; }
[ \t]           ;      /* ignore spaces */
\n              { return '\n'; }
.               { return yytext[0]; }

%%

int yywrap() { return 1; }
```

Exp3d.y:

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
int yyerror(const char *s);
%}

%token NUMBER

%%

input:
     /* empty */
   | input expr '\n'   { printf("Result = %d\n", $2); }
   ;
```

```
expr:
      expr '+' expr   { $$ = $1 + $3; }
    | expr '-' expr   { $$ = $1 - $3; }
    | expr '*' expr   { $$ = $1 * $3; }
    | expr '/' expr   { $$ = $1 / $3; }
    | '(' expr ')'    { $$ = $2; }
    | NUMBER          { $$ = $1; }
    ;

%%

int main() {
    printf("Enter expression: ");
    yyparse();
    return 0;
}

int yyerror(const char *s) {
    printf("Invalid expression!\n");
    return 0;
}
```

Output:


Activities
Terminal
Sep 26 21:29
cse@ubuntu: ~/4006/ex3
cse@ubuntu:~/4006/ex3$ flex ex3d.l
cse@ubuntu:~/4006/ex3$ yacc -d ex3d.y
ex3d.y: warning: 16 shift/reduce conflicts [-Wconflicts-sr]
cse@ubuntu:~/4006/ex3$ gcc lex.yy.c y.tab.c -o output -lfl
cse@ubuntu:~/4006/ex3$ ./output
Enter expression: 1+2
Result = 3
cse@ubuntu:~/4006/ex3$ ./output
Enter expression: 1*5+7
Result = 12
cse@ubuntu:~/4006/ex3$

Program:

Exp4.l:

```
%{
#include "ex4.tab.h"
#include <stdlib.h>
%}

%%

[0-9]+                { yylval.ival = atoi(yytext); return NUMBER; }
[a-zA-Z][a-zA-Z0-9]*     { yylval.sval = strdup(yytext); return ID; }
"="                   { return '='; }
"+"                   { return '+'; }
"*"                   { return '*'; }
";"                  { return ';'; }
[ \t\n]               ;   // ignore spaces
.                    { return yytext[0]; }

%%
int yywrap(){
return 1;
}
```

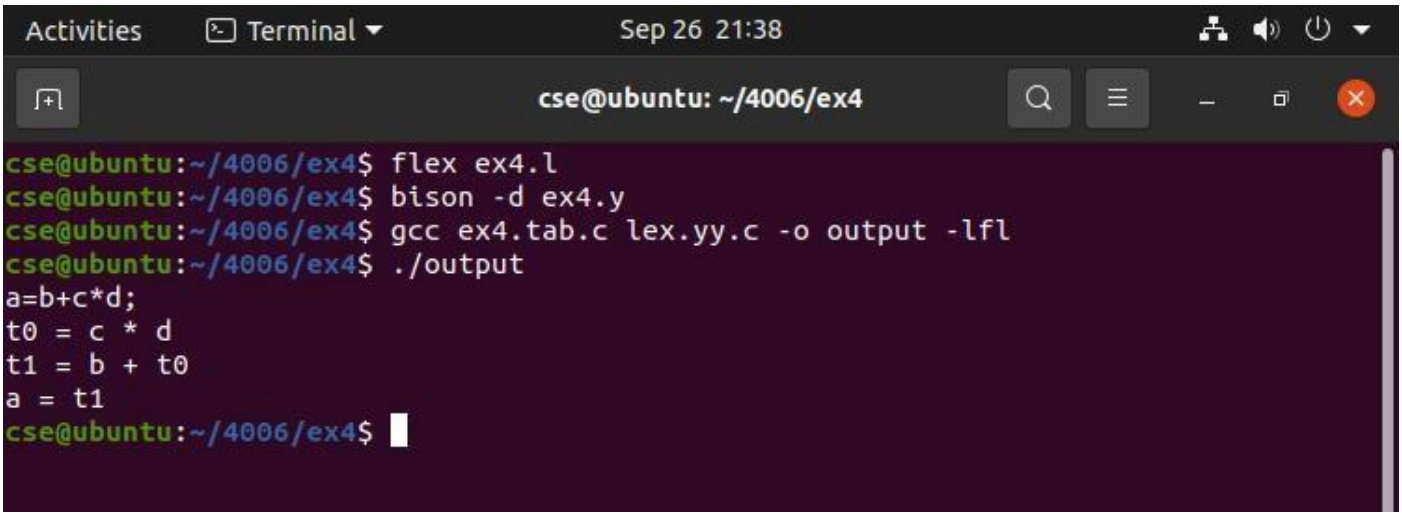
Exp4.y:

```
%{
#include   <stdio.h>
#include <stdlib.h>
#include <string.h>

int tempCount = 0;

char* newTemp() {
```

```
    static char buf[32];
    sprintf(buf, "t%d", tempCount++);
    return strdup(buf);
}

extern char* yytext;
int yylex(void);
int yyerror(const char *s);
%}

%union {
    int ival;
    char* sval;
}

%token <ival> NUMBER
%token <sval> ID
%type <sval> expr term factor

%left '+'
%left '*'

%%

stmt: ID '=' expr ';' {
    printf("%s = %s\n", $1, $3);
}
;

expr: expr '+' term {
    char* t = newTemp();
    printf("%s = %s + %s\n", t, $1, $3);
    $$ = t;
}
```

```
| term { $$ = $1; }
;

term: term '*' factor {
    char* t = newTemp();
    printf("%s = %s * %s\n", t, $1, $3);
    $$ = t;
}
| factor { $$ = $1; }
;

factor: ID     { $$ = $1; }
      | NUMBER {
          char buf[20];
          sprintf(buf, "%d", $1);
          $$ = strdup(buf);
      }
;
%%

int main() {
    return yyparse();
}

int yyerror(const char *s) {
    printf("Error: %s\n", s);
    return 0;
}
```

Output:


Activities
Terminal
Sep 26 21:38
cse@ubuntu: ~/4006/ex4
cse@ubuntu:~/4006/ex4$ flex ex4.l
cse@ubuntu:~/4006/ex4$ bison -d ex4.y
cse@ubuntu:~/4006/ex4$ gcc ex4.tab.c lex.yy.c -o output -lfl
cse@ubuntu:~/4006/ex4$ ./output
a=b+c*d;
t0 = c * d
t1 = b + t0
a = t1
cse@ubuntu:~/4006/ex4$

Program:

Exp5.l:

```
%{
#include "y.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%%

int             { return INT; }
[a-zA-Z][a-zA-Z0-9]*  { yylval.sval = strdup(yytext); return ID; }
[0-9]+          { yylval.sval = strdup(yytext); return NUMBER; }
"="             { return '='; }
";"             { return ';'; }
[ \t\n]+        ;   // ignore whitespace
.               { return yytext[0]; }

%%

int yywrap() { return 1; }
```

Exp5.y:

```
%{
#include   <stdio.h>
#include  <stdlib.h>
#include <string.h>

typedef struct {
    char* name;
    char* type;
} sym;
```

```
sym table[100];
int table_index = 0;

void insert(char* name, char* type) {
    table[table_index].name = strdup(name);
    table[table_index].type = strdup(type);
    table_index++;
}

char* lookup(char* name) {
    for(int i = 0; i < table_index; i++)
        if(strcmp(table[i].name, name) == 0) return table[i].type;
    return NULL;
}

extern char* yytext;
int yylex(void);
int yyerror(const char *s) {
    printf("Error: %s\n", s);
    return 0;
}
%}

%union { char* sval; }
%token <sval> ID NUMBER
%token INT
%type <sval> expr

%%

program: declarations statements ;

declarations:
     | declarations decl
```

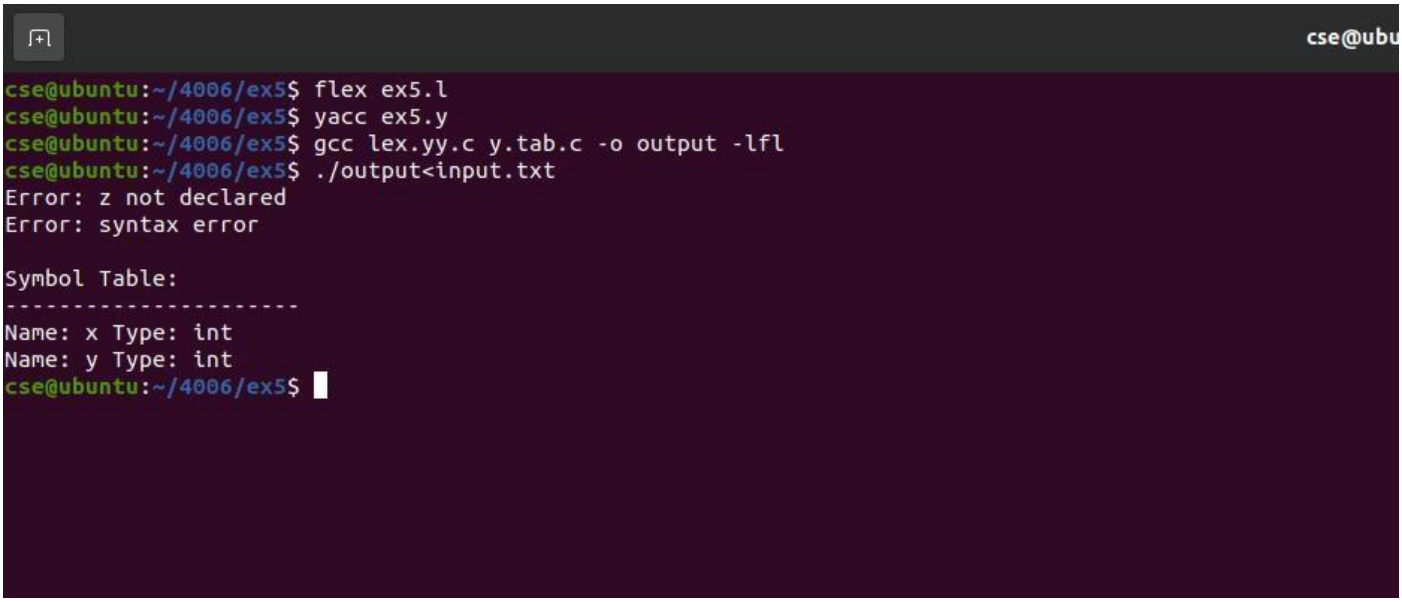
```
    ;

decl: INT ID ';' {
   insert($2,"int");
}
;

statements:
    | statements stmt
    ;

stmt: ID '=' expr ';' {
   char* t = lookup($1);
   if(t == NULL) printf("Error: %s not declared\n", $1);
   else if(strcmp(t,$3) != 0) printf("Type Error: %s and %s mismatch\n", $1, $3);
}
;

expr: NUMBER { $$ = "int"; }
   | ID {
      char* t = lookup($1);
      if(t == NULL) {
         printf("Error: %s not declared\n", $1);
         $$ = "int";
      } else $$ = t;
   }
;

%%

int main() {
   yyparse();
```

```
    printf("\nSymbol Table:\n");
    printf("------------------------\n");
    for(int i = 0; i < table_index; i++) {
        printf("Name: %s\tType: %s\n", table[i].name, table[i].type);
    }

    return 0;
}
```

input.txt:

```
int  x;
int  y;
x = 5;
y = x;
z = 10;
x = "Hello";
```

Output:



Program:

Exp7.c:

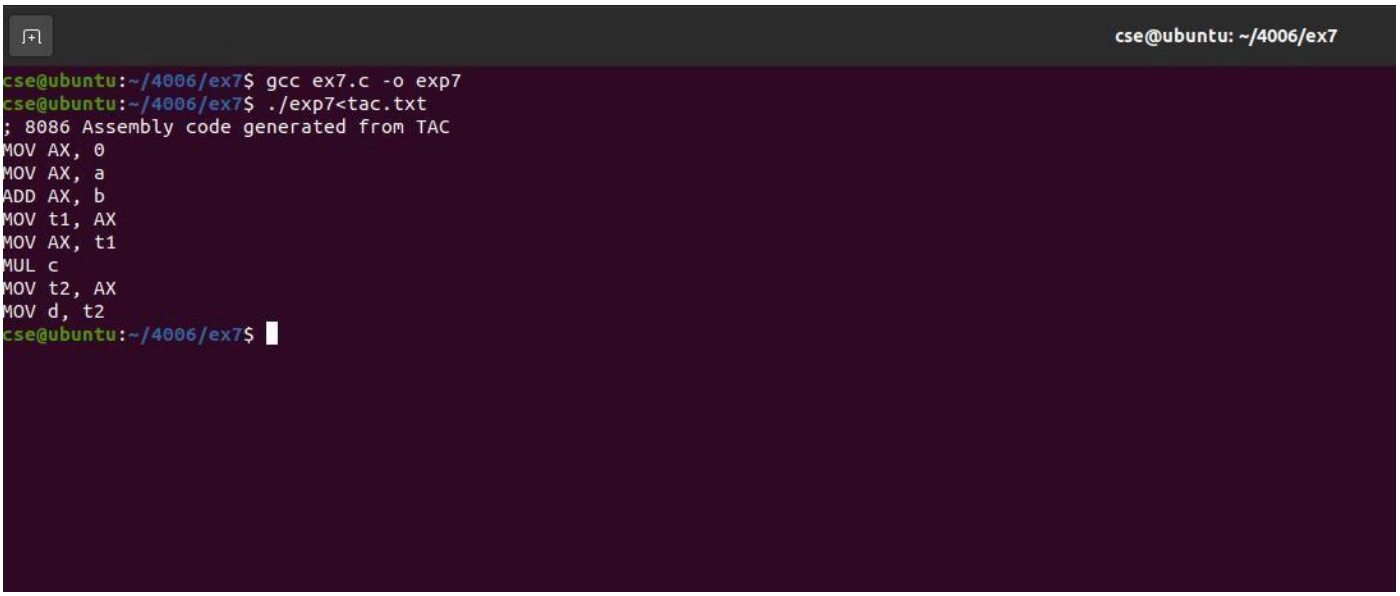
```
#include <stdio.h>
#include <string.h>

int main() {
   FILE *tacFile;
   char line[100], op[10], arg1[10], arg2[10], result[10];

   // Open TAC input file
   tacFile = fopen("tac.txt", "r");
   if (tacFile == NULL) {
      printf("Error: Cannot open TAC file\n");
      return 1;
   }

   printf("; 8086 Assembly code generated from TAC\n");
   printf("MOV AX, 0\n"); // initialize AX

   // Read TAC line by line
   while (fgets(line, sizeof(line), tacFile) != NULL) {
      if (sscanf(line, "%s = %s + %s", result, arg1, arg2) == 3) {
         printf("MOV AX, %s\n", arg1);
         printf("ADD AX, %s\n", arg2);
         printf("MOV %s, AX\n", result);
      } else if (sscanf(line, "%s = %s - %s", result, arg1, arg2) == 3) {
         printf("MOV AX, %s\n", arg1);
         printf("SUB AX, %s\n", arg2);
         printf("MOV %s, AX\n", result);
      } else if (sscanf(line, "%s = %s * %s", result, arg1, arg2) == 3) {
         printf("MOV AX, %s\n", arg1);
         printf("MUL %s\n", arg2);
```

```
            printf("MOV %s, AX\n", result);
        } else if (sscanf(line, "%s = %s / %s", result, arg1, arg2) == 3) {
            printf("MOV AX, %s\n", arg1);
            printf("DIV %s\n", arg2);
            printf("MOV %s, AX\n", result);
        } else if (sscanf(line, "%s = %s", result, arg1) == 2) {
            printf("MOV %s, %s\n", result, arg1);
        }
    }

    fclose(tacFile);
    return 0;
}
```

tac.txt:

```
t1 = a + b
t2 = t1 * c
d = t2
```

Output:



Program:

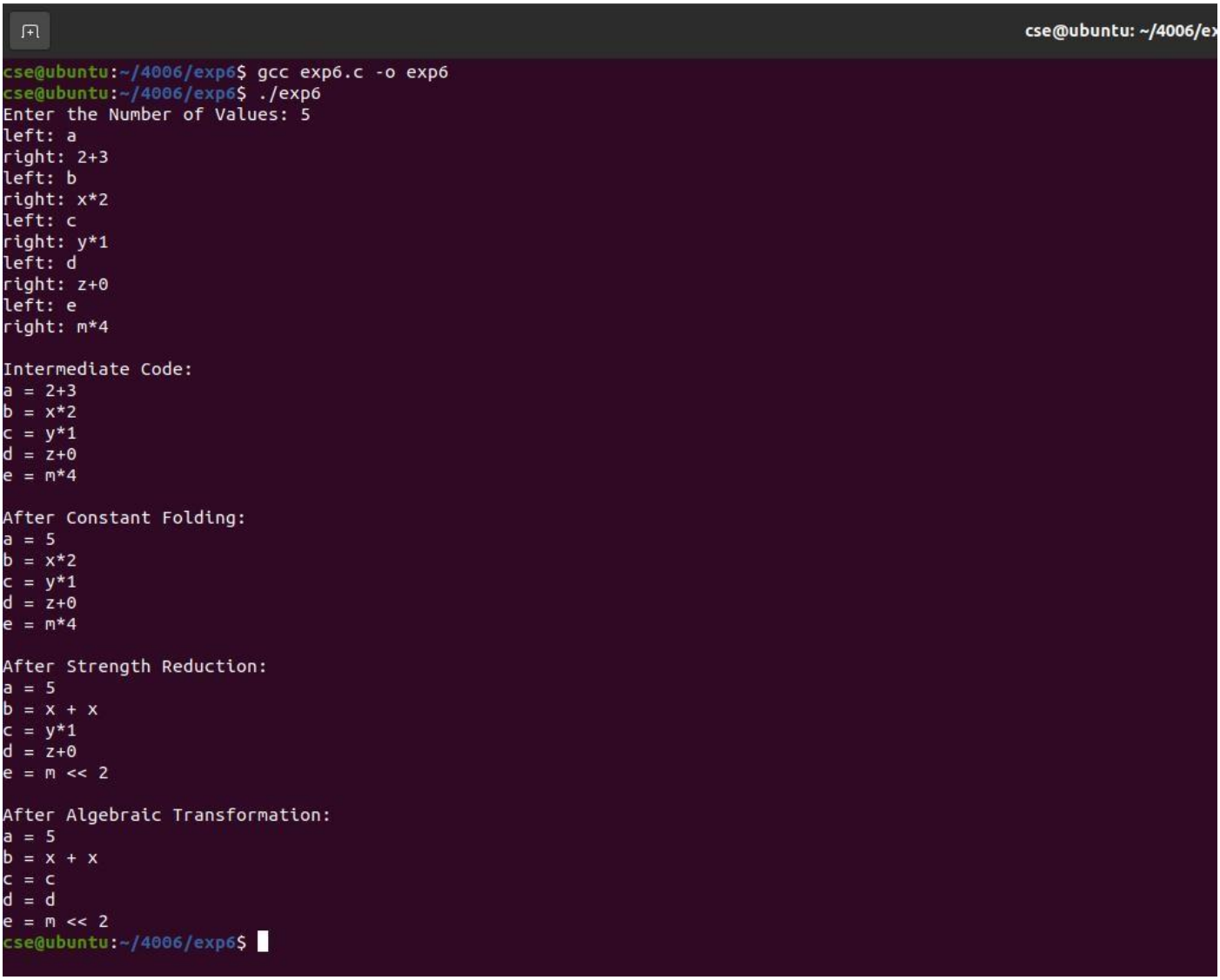
Exp6.c:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

struct op {
    char l;
    char r[20];
} op[10], pr[10];

int is_digit_expr(char *expr) {
    for (int i = 0; expr[i] != '\0'; i++) {
        if (!isdigit(expr[i]) && expr[i] != '+' && expr[i] != '-' && expr[i] != '*' && expr[i] != '/') {
            return 0;
        }
    }
    return 1;
}

void constant_folding(int i) {
    int a, b;
    if (sscanf(op[i].r, "%d+%d", &a, &b) == 2) {
        sprintf(op[i].r, "%d", a + b);
    } else if (sscanf(op[i].r, "%d*%d", &a, &b) == 2) {
        sprintf(op[i].r, "%d", a * b);
    } else if (sscanf(op[i].r, "%d-%d", &a, &b) == 2) {
        sprintf(op[i].r, "%d", a - b);
    } else if (sscanf(op[i].r, "%d/%d", &a, &b) == 2) {
        if (b != 0) {
            sprintf(op[i].r, "%d", a / b);
        }
```

```
    }
}
void strength_reduction(int i) {
    if (strstr(op[i].r, "*2")) {
        char var = op[i].r[0];
        sprintf(op[i].r, "%c + %c", var, var);
    } else if (strstr(op[i].r, "*4")) {
        char var = op[i].r[0];
        sprintf(op[i].r, "%c << 2", var);
    }
}
void algebraic_transformation(int i) {

    if (strstr(op[i].r, "*1")) {
        op[i].r[0] = op[i].l;
        op[i].r[1] = '\0';
    }

    else if (strstr(op[i].r, "+0")) {
        op[i].r[0] = op[i].l;
        op[i].r[1] = '\0';
    }

    else if (strstr(op[i].r, "*0")) {
        strcpy(op[i].r, "0");
    }
}

int main() {
    int a, i, j, n, z = 0, m, q;
    char *p, *l;
    char temp, t;
    char *tem;
```

```
    printf("Enter the Number of Values: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("left: ");
        scanf(" %c", &op[i].l);
        printf("right: ");
        scanf("%s", op[i].r);
    }
    printf("\nIntermediate Code:\n");
    for(i = 0; i < n; i++) {
        printf("%c = %s\n", op[i].l, op[i].r);
    }
    printf("\nAfter Constant Folding:\n");
    for(i = 0; i < n; i++) {
        if(is_digit_expr(op[i].r)) {
            constant_folding(i);
        }
        printf("%c = %s\n", op[i].l, op[i].r);
    }
    printf("\nAfter Strength Reduction:\n");
    for(i = 0; i < n; i++) {
        strength_reduction(i);
        printf("%c = %s\n", op[i].l, op[i].r);
    }
    printf("\nAfter Algebraic Transformation:\n");
    for(i = 0; i < n; i++) {
        algebraic_transformation(i);
        printf("%c = %s\n", op[i].l, op[i].r);
    }

    return 0;
}
```

Output:


cse@ubuntu: ~/4006/ex
cse@ubuntu:~/4006/exp6$ gcc exp6.c -o exp6
cse@ubuntu:~/4006/exp6$ ./exp6
Enter the Number of Values: 5
left: a
right: 2+3
left: b
right: x*2
left: c
right: y*1
left: d
right: z+0
left: e
right: m*4
Intermediate Code:
a = 2+3
b = x*2
c = y*1
d = z+0
e = m*4
After Constant Folding:
a = 5
b = x*2
c = y*1
d = z+0
e = m*4
After Strength Reduction:
a = 5
b = x + x
c = y*1
d = z+0
e = m << 2
After Algebraic Transformation:
a = 5
b = x + x
c = c
d = d
e = m << 2
cse@ubuntu:~/4006/exp6$